PID Control Theory and Implementation

ESP32 Motor Control Application

STAR Firmware Documentation

December 15, 2025

Contents

1 Introduction

A PID (Proportional-Integral-Derivative) controller is a feedback control mechanism widely used in industrial control systems. In the context of ESP32-based motor control, PID controllers enable precise closed-loop control by continuously calculating an error value as the difference between a desired setpoint and a measured process variable.

1.1 Control System Overview

The STAR firmware implements a discrete-time PID controller specifically designed for:

- Brushed DC motor speed control
- Position control with encoder feedback
- Temperature-compensated motor drive systems
- Battery-powered robotic applications

2 Mathematical Foundation

2.1 Continuous-Time PID Equation

The ideal continuous-time PID controller is defined by:

Continuous PID Equation

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \tag{1}$$

where:

- $u(t)$ = control output (e.g., motor PWM duty cycle)
- $e(t) = r(t) - y(t)$ = error signal
- $r(t)$ = setpoint (desired value)
- $y(t)$ = measured process variable (actual value)
- K_p = proportional gain
- K_i = integral gain
- K_d = derivative gain

2.2 Discrete-Time Implementation

For digital implementation on a microcontroller, the PID equation is discretized using numerical approximation:

Discrete PID Equation

$$u[k] = K_p e[k] + K_i \sum_{j=0}^{k} e[j]\Delta t + K_d \frac{e[k] - e[k-1]}{\Delta t} \tag{2}$$

where:

- k = discrete time step index
- Δt = sampling period (e.g., 0.001s for 1000Hz control loop)
- $e[k] = r[k] - y[k]$ = error at time step k

2.3 Component Analysis

2.3.1 Proportional Term

$$P[k] = K_p e[k] \tag{3}$$

Effect: Provides immediate corrective action proportional to the current error. Higher K_p increases system responsiveness but may cause overshoot and oscillation.

2.3.2 Integral Term

$$I[k] = K_i \sum_{j=0}^{k} e[j]\Delta t = K_i \left(I[k-1] + e[k]\Delta t\right) \tag{4}$$

Effect: Eliminates steady-state error by accumulating past errors. Higher K_i speeds up steady-state convergence but may cause overshoot and instability.

Anti-Windup: The integral term is clamped to prevent excessive accumulation:

$$I[k] = \text{clamp}(I[k], I_{\min}, I_{\max}) \tag{5}$$

2.3.3 Derivative Term

$$D[k] = K_d \frac{e[k] - e[k-1]}{\Delta t} \tag{6}$$

Effect: Provides damping by predicting future error based on rate of change. Reduces overshoot and improves stability, but is sensitive to noise.

3 Block Diagram

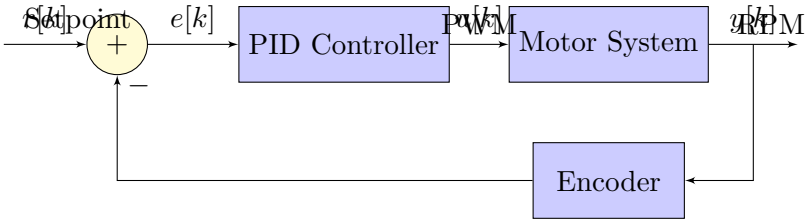

Figure 1: Closed-Loop PID Control System Block Diagram

The control loop operates as follows:

1. **Error Calculation:** $e[k] = r[k] - y[k]$
2. **PID Computation:** Calculate P, I, D terms
3. **Output Generation:** $u[k] = P[k] + I[k] + D[k]$
4. **Actuation:** Apply $u[k]$ to motor (PWM duty cycle)
5. **Measurement:** Read encoder position/velocity
6. **Feedback:** Use $y[k]$ for next iteration

4 PID Control Flow

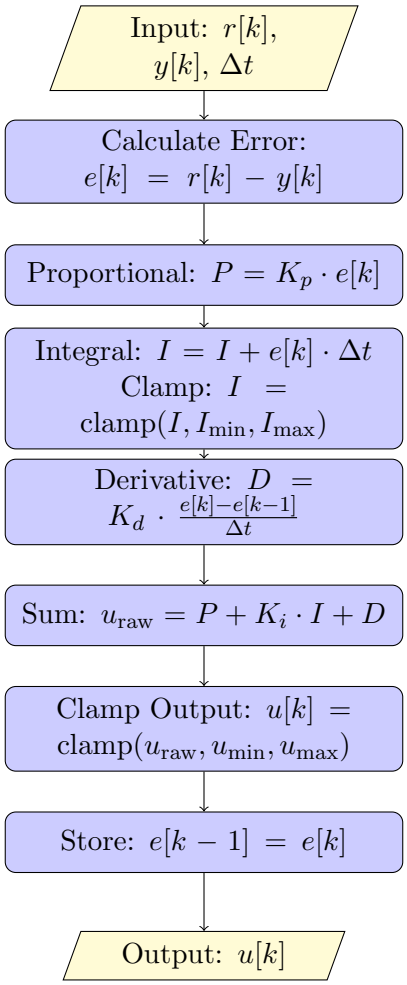

Figure 2: PID Algorithm Execution Flow

5 ESP32 Implementation Details

5.1 Data Structures

The STAR firmware uses two primary structures:

```
// Configuration structure (initialization)
typedef struct {
  float kp;            // Proportional gain
  float ki;            // Integral gain
  float kd;            // Derivative gain
  float output_min;    // Minimum output limit
  float output_max;    // Maximum output limit
  float integral_min;  // Anti-windup lower limit
  float integral_max;  // Anti-windup upper limit
} star_pid_config_t;

// Runtime handle (state preservation)
typedef struct {
  float kp, ki, kd;           // PID gains
  float output_min, output_max;
  float integral_min, integral_max;
  float integral;             // Accumulated integral
  float prev_error;          // Previous error (for derivative)
  bool initialized;          // Initialization flag
} star_pid_handle_t;
```

5.2 Initialization

```
star_pid_handle_t pid;
star_pid_config_t config = {
    .kp = 1.0f,
    .ki = 0.5f,
    .kd = 0.1f,
    .output_min = -100.0f,  // -100% duty cycle
    .output_max = 100.0f,   // +100% duty cycle
    .integral_min = -50.0f,
    .integral_max = 50.0f,
};
esp_err_t ret = star_pid_init(&pid, &config);
```

5.3 Control Loop Execution

Typical execution at 1000Hz (1ms sampling period):

```
float setpoint_rpm = 100.0f;
float measured_rpm = 95.0f;
float dt = 0.001f;  // 1ms

float output;
star_pid_compute(&pid, setpoint_rpm, measured_rpm, dt, &output);

// Apply output to motor driver
star_drv8243_set_speed(&motor, output);
```

6 Anti-Windup Mechanism

6.1 The Windup Problem

Integral windup occurs when the control output saturates (reaches $u_{\min}$ or $u_{\max}$) but the error remains non-zero. The integral term continues accumulating, leading to:

- Large overshoot when error changes sign
- Slow recovery from saturation
- System instability

6.2 Clamping Solution

The STAR implementation uses **integral clamping**:

$$I[k] = \begin{cases} I_{\max} & \text{if } I[k] > I_{\max} \\ I_{\min} & \text{if } I[k] < I_{\min} \\ I[k] & \text{otherwise} \end{cases} \tag{7}$$

This prevents the integral term from accumulating beyond practical limits, ensuring:

- Bounded integral accumulation
- Faster recovery from saturation
- Predictable transient response

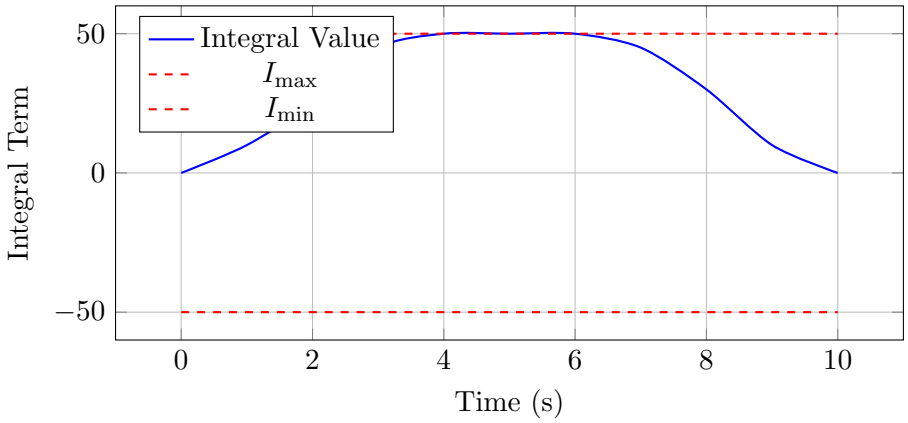

Figure 3: Integral Clamping Prevents Windup

7 Tuning Guidelines

7.1 Ziegler-Nichols Method

A classical tuning approach:

1. Set $K_i = K_d = 0$
2. Increase K_p until system oscillates
3. Record ultimate gain K_u and oscillation period T_u
4. Calculate gains:

Controller	K_p	K_i	K_d
P	$0.5K_u$	-	-
PI	$0.45K_u$	$1.2K_p/T_u$	-
PID	$0.6K_u$	$2K_p/T_u$	$K_pT_u/8$

Table 1: Ziegler-Nichols Tuning Parameters

7.2 Manual Tuning Process

1. **Start with P-only:** Set $K_i = K_d = 0$, increase K_p until acceptable response
2. **Add Integral:** Increase K_i to eliminate steady-state error
3. **Add Derivative:** Increase K_d to reduce overshoot and improve stability
4. **Fine-tune:** Adjust all three gains iteratively

7.3 Gain Effects Summary

Parameter	Rise Time	Overshoot	Steady-State Error
$K_p \uparrow$	Decrease	Increase	Decrease
$K_i \uparrow$	Decrease	Increase	Eliminate
$K_d \uparrow$	Minor change	Decrease	No effect

Table 2: Effect of Increasing PID Gains

8 Performance Characteristics

8.1 Stability Criteria

For a stable closed-loop system:

$$\text{Phase Margin} > 45^\circ \quad \text{and} \quad \text{Gain Margin} > 6 \text{ dB} \tag{8}$$

8.2 Step Response Specifications

- **Rise Time (t_r):** Time to reach 90% of setpoint
- **Settling Time (t_s):** Time to stay within $\pm 2\%$ of setpoint
- **Overshoot (M_p):** $M_p = \frac{y_{\max} - y_{ss}}{y_{ss}} \times 100\%$
- **Steady-State Error (e_{ss}):** $e_{ss} = \lim_{t \to \infty} |r(t) - y(t)|$

9 Motor Control Application

9.1 Complete System Example

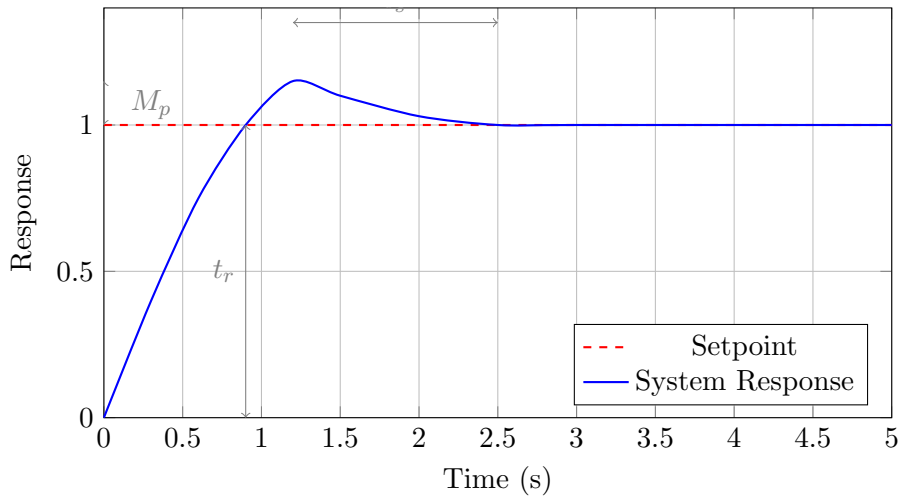

Figure 4: Typical Step Response Characteristics

```c
// Initialize motor driver (DRV8243)
star_drv8243_handle_t motor;
star_drv8243_config_t motor_cfg = {
    .bus_manager = &bus_manager,
    .pin_pwm_ph = GPIO_NUM_25,
    .pwm_freq_hz = 20000,  // 20 kHz PWM
};
star_drv8243_init(&motor, &motor_cfg);

// Initialize encoder
star_encoder_handle_t encoder;
star_encoder_config_t enc_cfg = {
    .pcnt_unit = PCNT_UNIT_0,
    .pin_a = GPIO_NUM_32,
    .pin_b = GPIO_NUM_33,
};
star_encoder_init(&encoder, &enc_cfg);

// Initialize PID
star_pid_handle_t pid;
star_pid_config_t pid_cfg = {
    .kp = 1.0f, .ki = 0.5f, .kd = 0.1f,
    .output_min = -100.0f, .output_max = 100.0f,
};
star_pid_init(&pid, &pid_cfg);

// Control loop (1000Hz)
while (1) {
    float velocity_rpm;
    star_encoder_get_velocity_rpm(&encoder, 1.0f, 500, &velocity_rpm);

    float output;
    star_pid_compute(&pid, 100.0f, velocity_rpm, 0.001f, &output);
    star_drv8243_set_speed(&motor, output);

    vTaskDelay(pdMS_TO_TICKS(1));
}
```

10 Advanced Topics

10.1 Derivative Filtering

To reduce noise sensitivity, the derivative term can be filtered:

$$D_{\text{filtered}}[k] = \alpha D[k] + (1 - \alpha) D_{\text{filtered}}[k-1] \tag{9}$$

where $\alpha \in [0, 1]$ is the filter coefficient.

10.2 Feed-Forward Control

For improved tracking performance, add feed-forward term:

$$u[k] = u_{\text{PID}}[k] + K_{ff} r[k] \tag{10}$$

10.3 Bumpless Transfer

When switching between manual and automatic mode, initialize integral term:

$$I[k] = \frac{u_{\text{manual}} - P[k] - D[k]}{K_i} \tag{11}$$

11 References

1. Astrom, K. J., & Hagglund, T. (2006). *Advanced PID Control*. ISA.

2. Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2019). *Feedback Control of Dynamic Systems* (8th ed.). Pearson.

3. ESP-IDF Documentation: Motor Control Peripheral (MCPWM)

4. STAR Firmware Documentation: `lib/star_pid/`

A Implementation Source Code

A.1 PID Compute Function

```
esp_err_t star_pid_compute(star_pid_handle_t* handle,
                           float setpoint,
                           float measured,
                           float dt,
                           float* output)
{
  // Validate inputs
  if (!handle || !output || dt <= 0.0f) {
    return ESP_ERR_INVALID_ARG;
  }

  // Calculate error
  float error = setpoint - measured;

  // Proportional term
  float p_term = handle->kp * error;

  // Integral term with anti-windup
```

```
    handle->integral += error * dt;
    handle->integral = clamp(handle->integral,
                             handle->integral_min,
                             handle->integral_max);
    float i_term = handle->ki * handle->integral;

    // Derivative term
    float derivative = (error - handle->prev_error) / dt;
    float d_term = handle->kd * derivative;

    // Compute total output
    float raw_output = p_term + i_term + d_term;

    // Clamp output to limits
    *output = clamp(raw_output,
                    handle->output_min,
                    handle->output_max);

    // Store error for next iteration
    handle->prev_error = error;

    return ESP_OK;
}
```

B Typical Parameter Ranges

Parameter	Typical Range	Motor Control Application
K_p	0.1 - 10.0	0.5 - 2.0 (velocity control)
K_i	0.01 - 5.0	0.1 - 1.0 (eliminate steady-state error)
K_d	0.001 - 1.0	0.01 - 0.2 (damping)
Δt	0.001 - 0.1 s	0.001 s (1000 Hz recommended)
Output Limits	System-dependent	± 100 (PWM duty cycle %)
Integral Limits	50-100% of output	± 50 (anti-windup)

Table 3: Typical PID Parameter Ranges